



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

**MAESTRÍA EN CIENCIA DE DATOS Y MÁQUINAS DE
APRENDIZAJE**

MATERIA:

Python para Ciencia de Datos

GRUPO 4:

Kevin Apolo
Sthefanny Chamorro
José Luis Criollo

Taller Práctico:

Semana 3

Fecha:

18 de agosto de 2026

Quito, Ecuador



TALLER PRÁCTICO 3

1. CONTEXTO

La organización donde trabaja María Fernanda produce informes sobre empleo, educación, movilidad y condiciones socioeconómicas a partir de archivos CSV, Excel y JSON descargados de portales oficiales y sistemas internos. En los últimos meses el volumen de datos ha aumentado y varios reportes se han retrasado por problemas de formatos, archivos duplicados y diferencias entre versiones. La institución desea profesionalizar sus flujos analíticos mediante automatización, entornos controlados, documentación y control de versiones.

El presente trabajo materializa ese rediseño sobre un caso concreto: la consolidación de las entregas académicas de dos campus —matriz y extensión— y su cruce con el catálogo maestro de estudiantes. Las tres fuentes reproducen las dificultades descritas: formatos heterogéneos, un archivo cuya extensión no corresponde a su contenido, registros sin correspondencia entre tablas y valores ausentes que exigen interpretación antes de ser tratados.

2. DESARROLLO

2.1 Estructura del proyecto y lectura parametrizada

2.1.1 Estructura de carpetas propuesta

[illegible]

2.1.2 Justificación de la arquitectura

- **Separación de datos originales y procesados.**

datos/originales/ se trata como una capa de solo lectura: ningún script ni notebook debe sobrescribir estos archivos. Esto permite que, si se detecta un error en el pipeline, el análisis se pueda re-ejecutar desde cero sin depender de una copia de respaldo externa. datos/procesados/ contiene únicamente artefactos generados por código (nunca editados a mano), de modo que su contenido es siempre reproducible a partir de datos/originales/ más el código en scripts/.

- **Separación entre lógica reutilizable (scripts/) y narrativa exploratoria (notebooks/).**

Las funciones de lectura, combinación y validación viven en scripts/ como módulos importables. Los notebooks las consumen en vez de reimplementarlas, evitando que la misma lógica de limpieza exista duplicada -y potencialmente inconsistente- en varias celdas. Esto también permite ejecutar el pipeline completo fuera de un notebook (por ejemplo, en un cron o pipeline de CI) sin depender de Jupyter.

- **reportes/ como salida obligatoria y separada de visualizaciones/.**

El reto exige indicadores de calidad cuantitativos (archivos procesados, registros leídos/descartados, claves sin correspondencia, nulos críticos, dimensiones de salida). Estos indicadores son texto/tablas, no gráficos, por lo que se separan de visualizaciones/ para que un revisor pueda auditar la calidad del proceso sin tener que abrir imágenes.

- **docs/ como memoria del criterio técnico.**

Cada decisión no evidente en el código (por qué un merge y no un concat, por qué un umbral de nulos es aceptable) se documenta aquí. El código responde qué se hizo; docs/ responde por qué, para que otro analista pueda auditar el razonamiento sin depender de instrucciones informales ni de preguntarle al autor original.

- **Trazabilidad de principio a fin.**

La secuencia de carpetas (originales → scripts → procesados → reportes/visualizaciones) refleja el orden real del flujo de datos. Un analista nuevo puede inferir el pipeline completo solo por la estructura de directorios, sin leer código previamente.

2.1.3 Estado de la estructura en el repositorio

Las siete carpetas descritas existen y están versionadas (con .gitkeep en las que aún no tienen archivos generados). Los tres archivos fuente provistos por la cátedra están ubicados en datos/originales/:

Archivo	Formato real	Registros
estudiantes_master.xlsx	CSV plano con extensión .xlsx (ver Hallazgo, sección 3.1)	60
entregas_campus_matriz.csv	CSV	150
entregas_campus_extension.csv	CSV	150

2.1.4 Lectura parametrizada de csv, excel y JSON

Implementación de referencia: scripts/lectura.py, con las funciones leer_entregas_csv(), leer_catalogo_maestro() y normalizar_json_anidado().

2.1.5 Extensión de archivo no confiable

Estudiantes_master.xlsx declara extensión .xlsx, pero su contenido real es texto CSV plano. Esto se comprobó inspeccionando los primeros bytes del archivo: un .xlsx real es un contenedor ZIP y empieza con la firma binaria PK; este archivo empieza directamente con "id_estudiante,nombre_completo,..." en texto plano. Un `pd.read_excel()` sin más falla o, peor, no falla pero interpreta mal el contenido.

Por esto, `leer_catalogo_maestro()` no confía en la extensión: intenta `pd.read_excel()` primero y, si lanza una excepción de formato (`ValueError/IOError`, el error que produce `openpyxl` al no encontrar la firma ZIP esperada), reintentará como CSV con los mismos parámetros de codificación y tipos. El resultado registra en `df.attrs["origen_lectura"]` si terminó leyéndose como Excel o como CSV, para que quede evidencia en el reporte de control de qué archivos requirieron la ruta alternativa.

Esta validación es deliberadamente defensiva: en un flujo real, un nuevo archivo del catálogo maestro podría llegar como .xlsx verdadero (con múltiples hojas) sin avisar, y el pipeline debe seguir funcionando sin intervención manual.

2.1.6 Parámetros de lectura y justificación

- **Codificación (encoding).**

Los tres archivos contienen tildes y ñes (nombres de estudiantes, nombres de materias). Se fuerza `encoding="utf-8"` explícitamente en lugar de dejar que pandas use el encoding por defecto del sistema operativo (`cp1252` en Windows), que corrompería esos caracteres de forma silenciosa: un tipo de error que no lanza excepción y por eso es peligroso si no se fija explícitamente.

- **Separador.**

Se confirmó por inspección directa que los tres archivos usan "," como separador de columnas. Se fija `sep=","` explícitamente en vez de dejarlo por defecto, para que el pipeline no dependa de que pandas siga adivinando correctamente si en el futuro se recibe un archivo con ";" (común en exportaciones regionales de Excel).

- **Tipos de columna.**

Todos los identificadores (`id_entrega`, `id_estudiante`) se leen como string, nunca como numérico, porque tienen prefijos alfabéticos (E001, PRJ-1001) y porque un ID nunca debe participar en operaciones aritméticas. Dejar que pandas infiera el tipo es riesgoso: si en algún archivo los IDs fueran puramente numéricos, pandas los leería como `int64` y un futuro merge con otra fuente que sí trae ceros a la izquierda ("007") fallaría silenciosamente.

puntaje_obtenido se tipa como float64, no int, porque debe tolerar valores nulos: las entregas con estado_entrega == "Pendiente" no tienen calificación. Forzar int en una columna con NaN no es posible en pandas sin una conversión adicional, así que float64 es el tipo mínimo que soporta el dato real sin perder información.

- **Tratamiento de fechas.**

Fecha_nacimiento y fecha_subida vienen en formato ISO 8601 (YYYY-MM-DD), por lo que `pd.read_csv(..., parse_dates=[...])` las interpreta sin ambigüedad y sin necesidad de un formato explícito. Se parsean en el momento de la lectura, no después, para que cualquier fecha malformada falle temprano, en la etapa de lectura, en vez de propagarse silenciosamente como texto hasta una etapa posterior del pipeline donde el error sería más difícil de rastrear.

2.1.7 Resumen de parámetros por archivo

Archivo	Delimitador	Encoding	Columnas fecha	Registros
entregas_campus_matriz.csv	,	UTF-8	fecha_subida	150
entregas_campus_extensio n.csv	,	UTF-8	fecha_subida	150
estudiantes_master.xlsx	, (texto plano)	UTF-8	fecha_nacimiento	60

2.2 Validaciones e integración

2.2.1 Verificación de estructura previa a la combinación

Antes de combinar cualquier fuente se comprueba que contenga las columnas obligatorias mediante `validar_columnas()`. Esta verificación responde a un criterio de costo: un fallo detectado en la lectura se corrige sobre la fuente original, mientras que el mismo fallo detectado tras consolidar obliga a rehacer el flujo completo.

La función interrumpe el proceso con un mensaje que nombra las columnas ausentes, en lugar de dejar que el error aparezca más adelante como un `KeyError` sin contexto.

2.2.2 Control de duplicados

El control distingue dos situaciones que no son equivalentes. Las filas completamente idénticas se eliminan porque representan la misma observación cargada dos veces. Las claves repetidas, en cambio, requieren interpretación: en una tabla de entregas un mismo estudiante aparece legítimamente varias veces, mientras que en el catálogo maestro la repetición de un identificador indicaría un problema de origen.

Por eso la validación de unicidad se aplica sobre `id_entrega` en las entregas y sobre `id_estudiante` en el catálogo, no de forma indiscriminada sobre ambas tablas. La función `controlar_duplicados()` devuelve el `DataFrame` depurado junto con el conteo de lo removido, de modo que la operación quede cuantificada.

2.2.3 Unicidad de la clave del catálogo

La verificación mediante `validar_clave_unica()` sobre el catálogo maestro es requisito previo del cruce relacional. Si esa clave no fuera única, el merge multiplicaría las filas de entrega y el total del reporte quedaría inflado sin que ningún error lo advirtiera. Comprobarlo antes convierte una suposición en una condición verificada.

2.3 Estrategia de combinación de datasets

2.3.1 Integración estructural mediante concat

Los dos archivos de entregas comparten exactamente la misma estructura de columnas, por lo que la operación correcta es apilarlos verticalmente con `pd.concat()` y no cruzarlos con `merge`. Un merge entre tablas de estructura idéntica no aporta columnas nuevas y solo introduce riesgo de duplicación.

El parámetro `ignore_index=True` reinicia el índice del resultado. Sin él, cada archivo conserva su índice original de 0 a 149 y el DataFrame consolidado queda con 150 índices duplicados. El objeto se construye sin lanzar error, pero cualquier operación posterior basada en el índice devolvería resultados ambiguos. Es un fallo silencioso: no avisa, solo corrompe.

La columna `campus_entrega` se inyecta como metadato de origen antes de concatenar, de modo que cada fila conserve la trazabilidad de qué archivo la aportó.

2.3.2 Integración relacional mediante merge con cardinalidad

El cruce entre las entregas consolidadas y el catálogo maestro responde a una relación asimétrica: un mismo estudiante puede registrar varias entregas durante el período, pero figura una sola vez en el catálogo. Por eso la cardinalidad declarada es `many_to_one`.

El parámetro `validate` no es documentación sino una prueba ejecutable: pandas verifica que `id_estudiante` sea único en la tabla derecha antes de cruzar. Si el catálogo tuviera un estudiante duplicado, el merge falla de forma explícita en lugar de multiplicar las filas de entrega en silencio.

Se emplea `how="left"` para conservar todas las entregas aunque el estudiante no exista en el catálogo. Un `inner join` las habría descartado y el total del reporte habría bajado de 300 a 295 sin dejar rastro del descarte. El parámetro `indicator` marca el origen de cada fila, lo que permite cuantificar las claves sin correspondencia.

La operación `join` no se emplea porque actúa sobre el índice y no sobre una columna clave. Usarla exigiría reindexar ambas tablas por `id_estudiante`, un paso innecesario cuando `merge` resuelve el cruce directamente sobre la columna.

2.3.3 Verificación posterior al cruce

Tras el merge se compara el número de filas contra el original. Si difiere, la función `cruzar_maestro()` interrumpe el proceso. Esta comparación detecta la multiplicación indebida, que es el fallo más difícil de advertir a simple vista porque produce un DataFrame perfectamente válido con totales incorrectos.

2.3.4 Claves sin correspondencia

El cruce identificó cinco entregas cuyo estudiante no consta en el catálogo maestro, correspondientes a los identificadores E555, E666, E777, E888 y E999. El patrón de la numeración sugiere registros de prueba o de contingencia antes que estudiantes reales.

Aun así estos registros no se eliminan. Corresponden a entregas efectivamente registradas en el sistema y descartarlas alteraría el conteo del reporte académico. Se conservan marcados mediante la bandera estudiante_identificado, de modo que el área que administra el catálogo pueda resolver su origen. La decisión sobre su validez compete a esa área y no al proceso de consolidación.

2.3.5 Valores nulos con causa identificable

La columna puntaje_obtenido presenta nueve valores nulos. Antes de decidir su tratamiento se analizó su distribución mediante una tabla cruzada contra el estado de la entrega.

El resultado muestra que los nueve nulos corresponden exactamente a las entregas en estado "Pendiente": entregas aún no calificadas, cuya ausencia de puntaje es estructuralmente correcta. Las quince entregas en estado "Revisión" sí tienen puntaje, lo que confirma que la relación es específica y no casual.

En consecuencia estos valores no se imputan ni se eliminan. Imputarlos con la media inventaría calificaciones inexistentes; eliminarlos borraría entregas reales del conteo. Se excluyen del cálculo de promedios pero se conservan en el total, marcados mediante la columna puntaje_pendiente.

2.4 Automatización del flujo

2.4.1 Rutas relativas y ejecución sin intervención manual

Para garantizar que el flujo sea completamente reproducible y ejecutable sin intervención manual, se implementó el manejo dinámico de rutas mediante la librería nativa 'pathlib'. Se desarrolló la función localizar_raiz(), la cual escanea e identifica automáticamente el directorio base del proyecto, permitiendo la lectura y escritura correcta sin importar la ruta específica del entorno donde se ejecute.

Adicionalmente, el código crea de manera automatizada las carpetas de salida necesarias (procesados, reportes, visualizaciones) si estas no existen. También parametriza de manera centralizada las fechas de ejecución mediante las variables fecha_proceso y sello temporal, eliminando por completo la necesidad de actualizar variables 'hardcodeadas' en cada corrida.

2.4.2 Funciones reutilizables y separación de responsabilidades

La lógica del flujo reside en dos módulos importables dentro de scripts/. El módulo lectura.py concentra la ingesta de las fuentes con sus parámetros declarados, y funciones.py agrupa la validación, la integración y el control de calidad. Cada módulo tiene una responsabilidad única: uno lee, el otro verifica y combina.

Esta separación evita que la misma lógica exista duplicada en varias celdas del cuaderno. Si cambia un criterio de lectura o una regla de validación, se corrige en un solo lugar y todo el



flujo se actualiza. Además permite ejecutar el pipeline fuera de un notebook, por ejemplo desde un proceso programado, sin depender de Jupyter.

2.4.3 Procesamiento iterativo de fuentes

Las dos fuentes de entregas se recorren mediante un ciclo sobre un diccionario que asocia cada campus con su ruta. Incorporar un tercer campus no exige duplicar celdas sino agregar una entrada a ese diccionario. El ciclo alimenta simultáneamente una bitácora de lectura que registra fuente, origen, número de registros y columnas, insumo directo del reporte de calidad.

2.4.4 Variables derivadas y agregaciones

A partir del dataset consolidado se generan variables temporales, el indicador de aprobación y la marca de puntaje pendiente. Sobre esa base se construyen dos agregaciones: una por campus, que incluye la desviación estándar para contextualizar el promedio, y otra por materia ordenada por rendimiento. Ambas se exportan como archivos independientes, de modo que el tablero no dependa de recalcularlas.

2.5 Indicadores de calidad del proceso

El flujo cierra con un reporte de control que cuantifica el resultado del procesamiento. Los indicadores se exportan a reportes/ como archivo tabular, separados de las visualizaciones, para que un revisor pueda auditar la calidad del proceso sin abrir imágenes.

2.5.1 Archivos procesados: 3

Los dos archivos de entregas y el catálogo maestro, todos leídos correctamente pese a la discrepancia de formato del último.

2.5.2 Registros leídos: 360

Suma de 150 entregas del campus matriz, 150 del campus extensión y 60 estudiantes del catálogo.

2.5.3 Registros descartados: 0

No se detectaron filas completamente duplicadas en ninguna de las fuentes. El indicador se reporta aunque su valor sea cero, porque su ausencia también es información: confirma que el control se ejecutó.

2.5.4 Claves sin correspondencia: 5

Entregas cuyo estudiante no figura en el catálogo maestro, conservadas y marcadas según lo expuesto en 2.3.4.

2.5.5 Nulos críticos: 0

Se consideran críticos únicamente los nulos que impedirían procesar el registro, es decir, los presentes en la clave de cruce o en la fecha de subida. No se detectó ninguno.

2.5.6 Nulos esperados: 9

Corresponden a las entregas pendientes de calificación. Se reportan por separado de los críticos porque su ausencia tiene una causa identificable y no constituye un defecto de calidad del dato. Esta distinción es el aporte metodológico central del reporte: contabilizarlos como defecto habría llevado a intervenir sobre datos que no lo requieren.

2.5.7 Dimensiones de salida: 300 filas por 20 columnas

Las 300 entregas consolidadas se conservan íntegras. Las columnas incluyen las variables originales, las aportadas por el catálogo y las derivadas por el proceso.

2.5.8 Archivos generados: 6

Un dataset consolidado, dos resúmenes agregados, una visualización, el reporte de indicadores y la bitácora de lectura.

2.6 Reproducibilidad técnica

2.6.1 Entorno virtual y archivo de dependencias

El proyecto declara sus dependencias en requirements.txt con versiones fijadas. Esto responde a un problema concreto: un flujo que funciona con una versión de pandas puede comportarse distinto con otra, y sin un registro explícito la diferencia es indetectable. El entorno virtual aísla esas dependencias del resto del sistema, evitando que la actualización de otro proyecto altere este.

La versión de Python se declara por separado en el README, porque pip gestiona librerías pero no el intérprete. Una dependencia puede no instalarse siquiera si la versión del intérprete es incompatible.

2.6.2 Ejecución en Jupyter o Colab

El cuaderno está diseñado para ejecutarse en ambos entornos. La localización dinámica de la raíz permite que funcione tanto desde la carpeta notebooks/ como desde la raíz del proyecto, sin rutas absolutas que aten el código a una máquina específica. El README documenta las dos rutas de ejecución.

2.6.3 Control de versiones con Git

El proyecto se versiona en un repositorio de GitHub con dos ramas. La rama main conserva la versión final y estable; la rama desarrollo acumula el trabajo en curso de los tres integrantes. La integración a main se realiza al cierre, lo que deja registro visible de qué cambios se incorporaron y cuándo.

Las carpetas que aún no contienen archivos generados se versionan mediante un archivo .gitkeep, ya que Git no registra directorios vacíos. El archivo .gitignore excluye el entorno virtual y los archivos temporales del intérprete, que se regeneran localmente y no deben versionarse.

2.6.4 Trazabilidad del flujo

La capa datos/originales/ se trata como zona inmutable: ningún script escribe sobre ella. Las salidas se versionan con un sello de fecha en el nombre del archivo, de modo que cada ejecución quede registrada en lugar de sobrescribir la anterior. Si se detecta un error en una regla de limpieza, basta con regenerar las capas siguientes desde los datos originales sin volver a solicitar las fuentes.

3. CONCLUSIONES

- El rediseño demuestra que la confiabilidad de un flujo analítico no depende de las herramientas empleadas sino de las verificaciones que incorpora. Los tres problemas más relevantes detectados —un archivo cuya extensión no corresponde a su contenido, índices que se duplican al apilar fuentes y registros sin correspondencia entre tablas— comparten una característica que los vuelve peligrosos: ninguno produce un error visible. El código se ejecuta, los objetos se construyen y el reporte se genera. El daño aparece cuando alguien compara cifras o toma una decisión sobre totales incorrectos.
- La estructura en capas y la separación entre lógica reutilizable y narrativa exploratoria hacen que el proceso sea auditable de principio a fin. La zona de datos originales permanece inmutable, las funciones residen en módulos importables y cada salida se versiona con sello temporal. Un analista externo puede reconstruir qué datos alimentaron cada resultado sin depender de explicaciones informales, que era precisamente la carencia del flujo anterior.
- El tratamiento de los valores ausentes evidencia que ciertas decisiones aparentemente técnicas no lo son. Los nueve puntajes nulos coinciden exactamente con las entregas en estado "Pendiente", de modo que su ausencia es estructuralmente correcta y no un defecto de calidad. Distinguir entre nulo crítico y nulo esperado exigió analizar el patrón antes de intervenir; aplicar una regla general de imputación o eliminación habría producido un dataset formalmente limpio pero analíticamente falso.

Ningún registro fue eliminado durante el proceso. Las cinco entregas sin estudiante en el catálogo y los nueve puntajes pendientes se conservaron marcados mediante banderas explícitas, trasladando al consumidor del reporte la información necesaria para calibrar su confianza. Marcar en lugar de descartar preserva la posibilidad de auditar y evita que la limpieza oculte problemas que corresponde resolver en la fuente.

Finalmente, la reproducibilidad se materializa en elementos concretos y verificables: rutas relativas que no atan el código a una máquina, dependencias declaradas con versiones fijadas, control de versiones con ramas diferenciadas para trabajo y entrega, y un reporte de indicadores que cuantifica el resultado de cada ejecución. El flujo dejó de ser un prototipo que funciona en un equipo para convertirse en un proceso que puede repetirse, revisarse y sostenerse en el tiempo.



4. NOTEBOOK

<https://github.com/jose264/TallerSemana03>